

# Programming in java

## Project Report



Name : Gauri Gupta

Registration no. : 25BAI11091

Branch : CSE(AIML)

Course Code : CSE2006

# INTRODUCTION

Course registration is one of the most time-sensitive processes in any academic institution. When registration opens, large numbers of students often attempt to secure a seat in the same popular course within seconds of each other. If a system processes these requests without proper safeguards, it can allocate the same seat to more than one student — a classic race condition.

This project, the Concurrent Course Registration & Seat Allocation System, is a Java console application built to demonstrate how core Java concepts — object-oriented design, exception handling, multithreading, collections, and JDBC — come together to solve this real problem. Multiple students are simulated as concurrent threads competing for a limited number of seats, and the system guarantees, through synchronized access, that no seat is ever double-booked.

Beyond the concurrency logic, the system also persists registration data using JDBC (SQLite) and exports registration/seat-availability reports using file I/O, giving a complete, end-to-end demonstration of the syllabus modules covered in this course.

# PROBLEM STATEMENT

College course registration periods often involve hundreds of students trying to register for a limited number of seats in popular courses within a very short window of time — often within seconds of registration opening. When multiple registration requests are processed at the same time without proper safeguards, it can lead to race conditions: for example, two students could both be allocated the same last seat, resulting in over-enrollment, inconsistent seat counts, and an unreliable system.

This project solves that problem by simulating multiple students registering for the same course simultaneously (using Java threads) and ensuring — through synchronized, thread-safe logic — that:

- No seat is ever allocated to more than one student

- Students who miss out on a seat are automatically placed on a waitlist

- A freed seat is automatically promoted to the next waitlisted student

- The correctness of this concurrency handling is verified through automated unit tests that simulate 20 students competing for 3 seats, confirming that exactly 3 registrations succeed every time, regardless of thread execution order.

# Target Users

- Students — represented as concurrent registration requests (simulated via threads) attempting to secure a seat in a course
- Administrators — responsible for defining courses and the number of available seats
- College evaluators / instructors — reviewing the project for correct, real-world application of Java concurrency, OOP, exception handling, collections, and JDBC

# Scope

In scope:

- Defining courses with a fixed number of seats
- Concurrent student registration via multithreading
- Thread-safe seat allocation using Java's synchronized keyword
- Waitlist management with automatic promotion
- Custom exception handling for duplicate registrations and unavailable seats
- Persistence via JDBC (SQLite) and report export via file I/O
- Automated JUnit testing of the concurrency guarantees

# FUNCTIONAL REQUIREMENTS

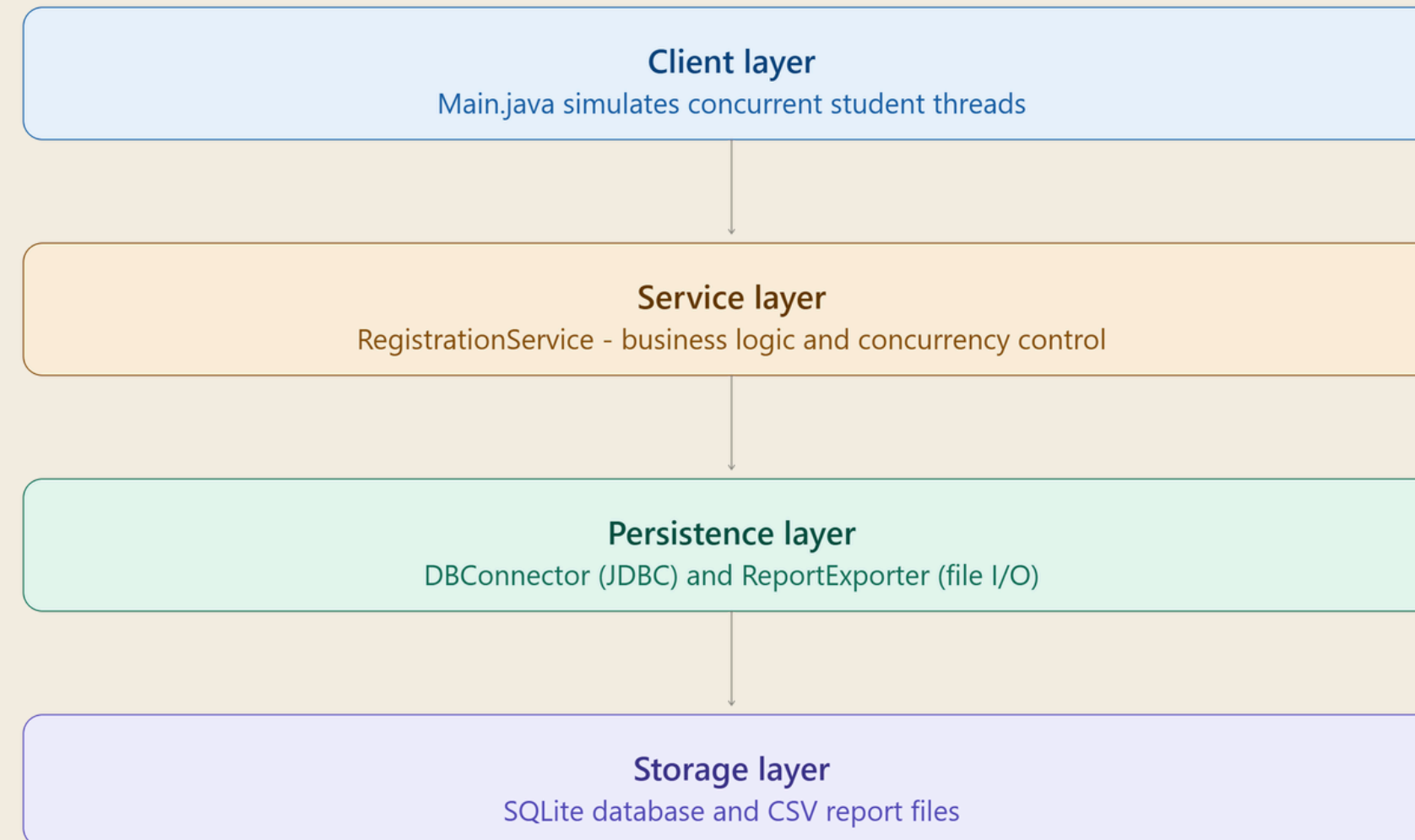
| Module                         | Description                                                                                                                              |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| User Management                | Represents students and admins interacting with the system (Student, Admin classes).                                                     |
| Course & Seat Management       | Admins define courses with a fixed number of seats; seat status can be viewed at any time.                                               |
| Concurrent Registration Engine | Multiple students (threads) can attempt to register for the same course at once; seat allocation is thread-safe via synchronized blocks. |
| Waitlist Management            | Students who miss out on a seat are placed on a waitlist and auto-promoted when a seat frees up.                                         |
| Reporting                      | Registration and seat-availability data is exported as CSV reports using file I/O.                                                       |

# NON-FUNCTIONAL REQUIREMENTS

1. Performance Concurrent registration requests are processed efficiently using a fixed thread pool (ExecutorService) so many simultaneous requests don't degrade responsiveness.
2. Reliability Seat allocation is guaranteed correct under concurrent access — verified by JUnit tests where 20 threads race for 3 seats and exactly 3 succeed every run.
3. Scalability The course catalog is stored in a Map, allowing the system to handle a growing number of courses and students without structural changes.
4. Error Handling Strategy Custom checked exceptions (SeatUnavailableException, DuplicateRegistrationException) ensure failure cases are handled explicitly.
5. Maintainability The codebase is organized into clear packages (model, service, exception, db, util).
6. Resource Efficiency Database connections and file writers are managed using try-with-resources to avoid resource leaks

# SYSTEM ARCHITECTURE

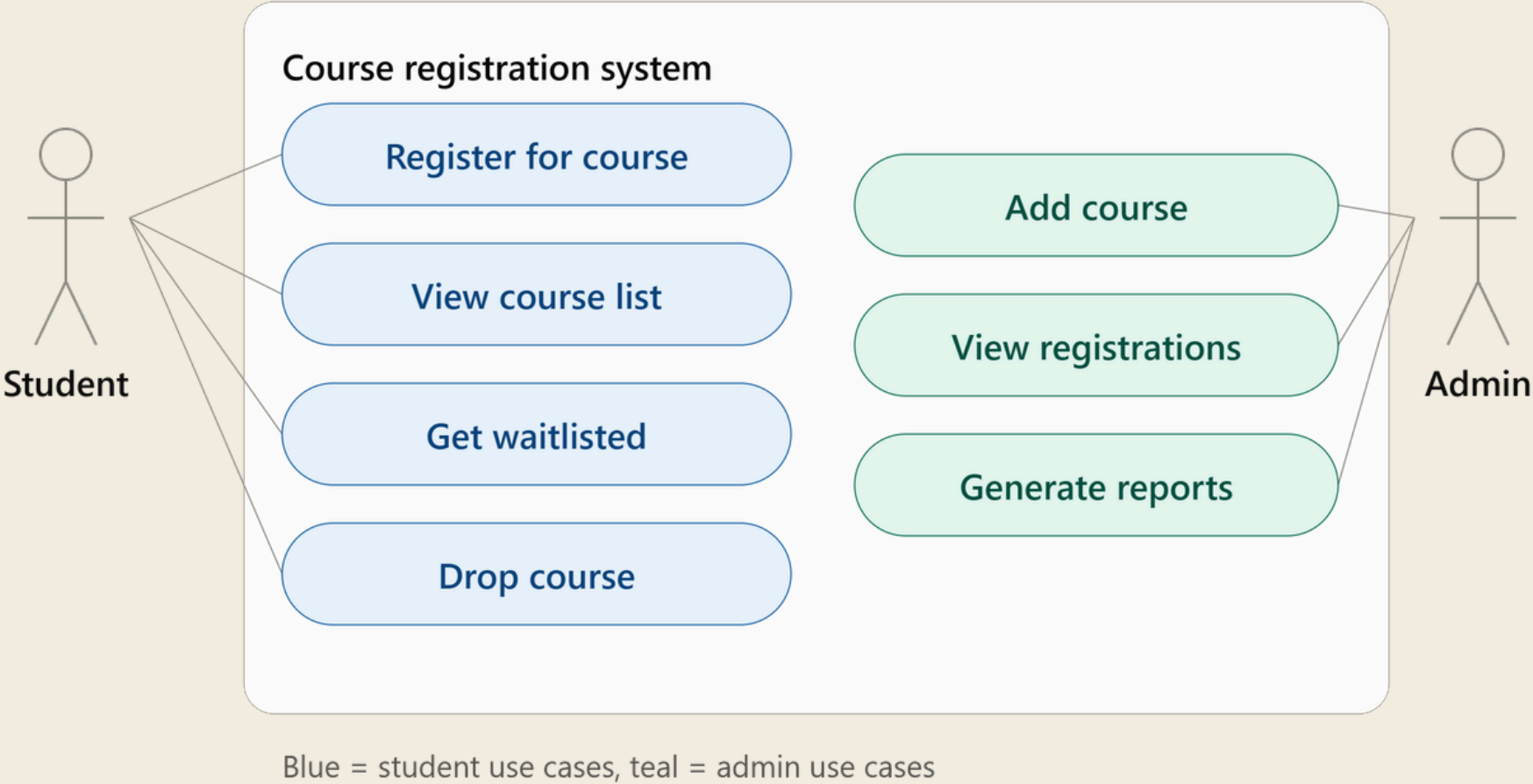
The system follows a simple layered architecture that separates concerns between simulating client requests, business logic, persistence, and storage.



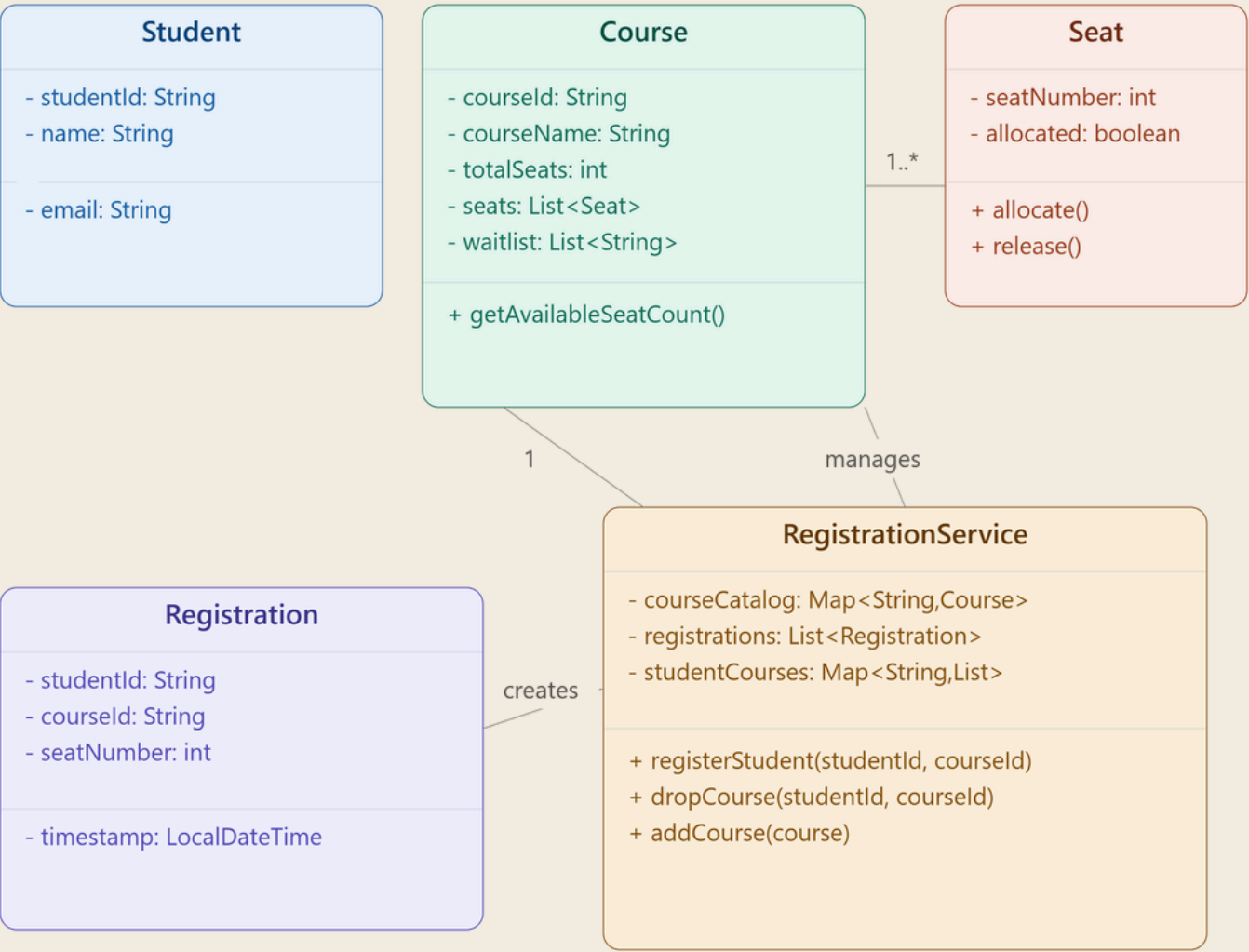
The Client layer (Main.java) simulates multiple students registering concurrently. The Service layer (RegistrationService) contains the core business logic and concurrency control. The Persistence layer bridges the application to durable storage via JDBC and file I/O, and the Storage layer holds the actual SQLite database and CSV report files.

# DESIGN DIAGRAM

## 1.Use Case Diagram



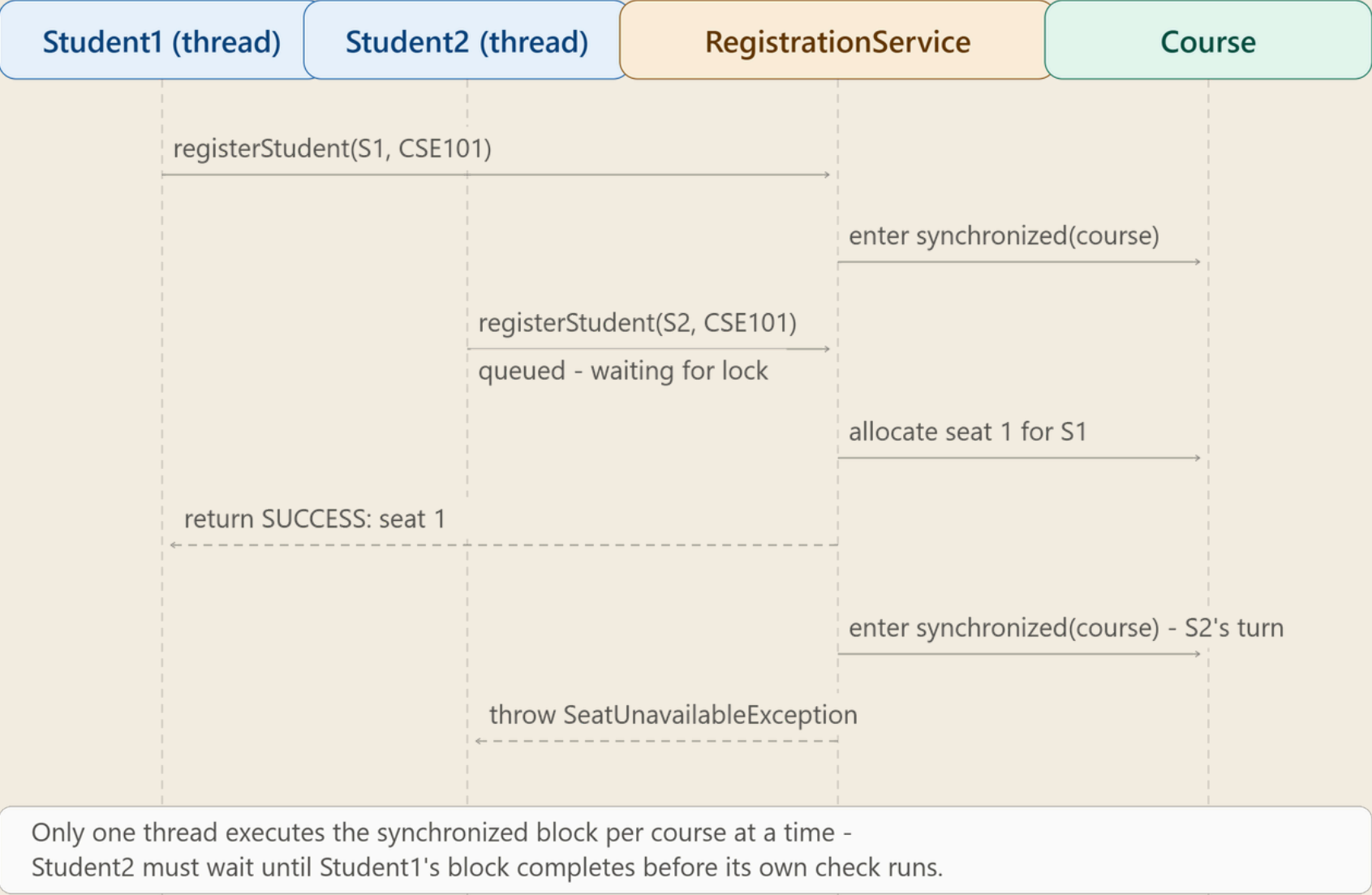
## 2. Class Diagram



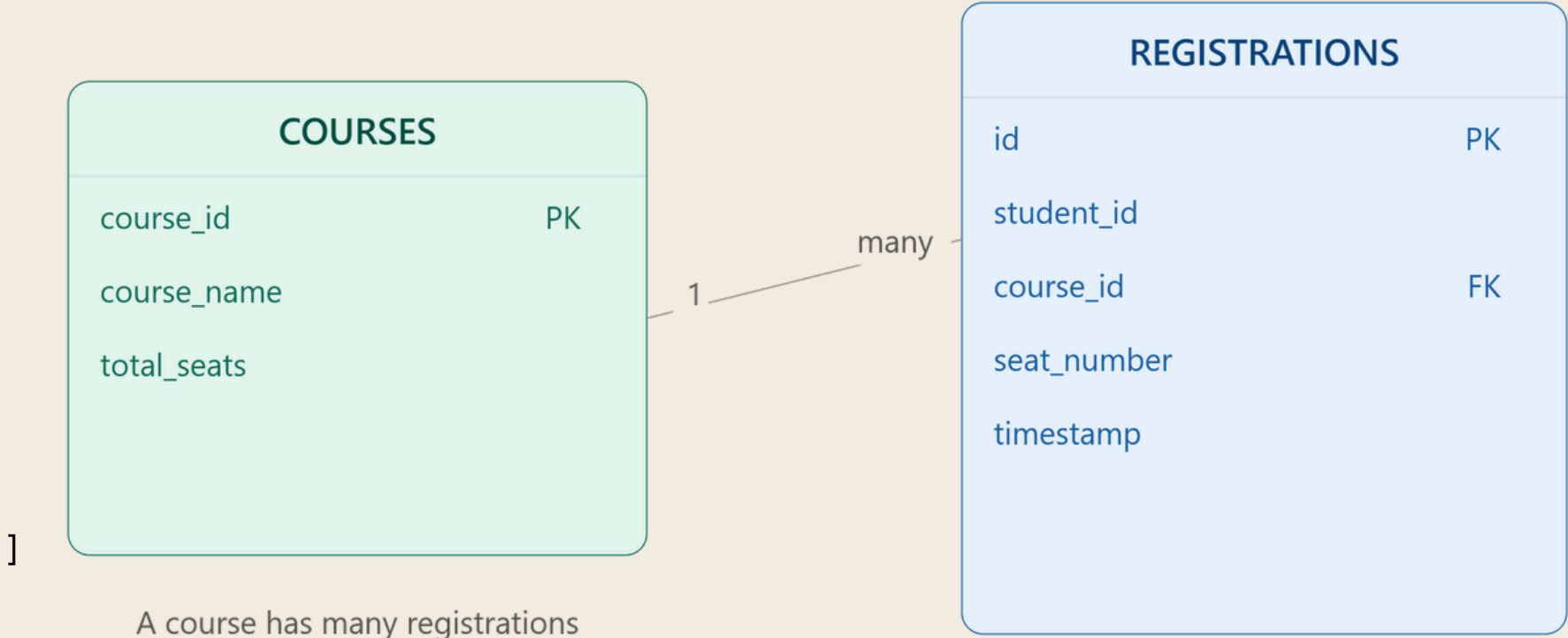
Relationships: Course has many Seats; RegistrationService manages Courses and creates Registrations

# DESIGN DIAGRAMS

## 3. Sequence Diagram



## 4. ER Diagram



# DESIGN DECISIONS & RATIONALE

- Synchronizing on the Course object (rather than a single global lock) — allows registration attempts for different courses to proceed in parallel, while still serializing access to any one course's seats.
- Using checked exceptions (SeatUnavailableException, DuplicateRegistrationException) — forces calling code to explicitly handle failure cases instead of allowing silent bugs.
- Using an ExecutorService thread pool instead of raw Thread objects — makes it easy to simulate a configurable number of concurrent students and cleanly wait for all of them to finish (awaitTermination).
- Storing the waitlist as a List<String> on the Course itself — keeps waitlist state co-located with the course it belongs to, simplifying auto-promotion logic on drop.
- Choosing SQLite over a full database server — removes the need for external database setup, so the project runs immediately on any machine.
- Separating persistence (DBConnector) from reporting (ReportExporter) — keeps JDBC logic and file I/O logic independent and individually testable

# IMPLEMENTATION DETAILS

The project is built with Maven and organized into the following packages:

- model — Student, Admin, Course, Seat, Registration: plain data-holding classes with encapsulated fields.
- exception — SeatUnavailableException, DuplicateRegistrationException: custom checked exceptions.
- service — RegistrationService (core business logic and concurrency control) and StudentRegistrationTask (a Runnable representing one student's registration attempt on its own thread).
- db — DBConnector: manages the JDBC connection to a local SQLite database and auto-creates the required tables.
- util — ReportExporter: writes registration and seat-availability data to CSV files using BufferedWriter.

# SCREENSHOTS

## 1. Concurrent Registration Simulation

Six student threads attempt to register for a 3-seat course. Exactly three succeed; the rest are automatically waitlisted. A drop and automatic waitlist promotion is also demonstrated.

```
• === Simulating concurrent registration for CSE101 (3 seats, 6 students) ===
pool-1-thread-2 -> WAITLISTED: No seats left in CSE101. Student S2 added to waitlist.
pool-1-thread-3 -> WAITLISTED: No seats left in CSE101. Student S3 added to waitlist.
pool-1-thread-4 -> WAITLISTED: No seats left in CSE101. Student S4 added to waitlist.
pool-1-thread-6 -> SUCCESS: Registration{student=S6, course=CSE101, seat=2, at=2026-09-06T03:38:30.645416300}
pool-1-thread-1 -> SUCCESS: Registration{student=S1, course=CSE101, seat=1, at=2026-09-06T03:38:30.644413600}
pool-1-thread-5 -> SUCCESS: Registration{student=S5, course=CSE101, seat=3, at=2026-09-06T03:38:30.645416300}

Final seat status for CSE101: Course{CSE101 - Core Java Programming, availableSeats=0/3}
Waitlist for CSE101: [S4, S2, S3]

=== S1 drops CSE101 ===
Seat status after drop: Course{CSE101 - Core Java Programming, availableSeats=0/3}
Waitlist after promotion: [S2, S3]
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.

All registrations saved to database (registration.db)
Registration report written to registration_report.csv
Seat availability report written to seat_availability_report.csv
```

## 2. Automated Test Results

Twenty student threads race for a 3-seat course under JUnit. The tests confirm exactly 3 registrations succeed, with zero failures and zero errors.

```
--- surefire:3.2.5:test (default-cli) @ course-registration-system ---
Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider

-----
T E S T S
-----
Running com.registration.RegistrationServiceTest
pool-2-thread-9 -> WAITLISTED: No seats left in TEST101. Student S9 added to waitlist.
pool-2-thread-6 -> WAITLISTED: No seats left in TEST101. Student S6 added to waitlist.
pool-2-thread-4 -> WAITLISTED: No seats left in TEST101. Student S4 added to waitlist.
pool-2-thread-7 -> WAITLISTED: No seats left in TEST101. Student S7 added to waitlist.
pool-2-thread-10 -> WAITLISTED: No seats left in TEST101. Student S10 added to waitlist.
pool-2-thread-5 -> WAITLISTED: No seats left in TEST101. Student S5 added to waitlist.
pool-2-thread-11 -> WAITLISTED: No seats left in TEST101. Student S11 added to waitlist.
pool-2-thread-13 -> WAITLISTED: No seats left in TEST101. Student S13 added to waitlist.
pool-2-thread-12 -> WAITLISTED: No seats left in TEST101. Student S12 added to waitlist.
pool-2-thread-8 -> WAITLISTED: No seats left in TEST101. Student S8 added to waitlist.
pool-2-thread-14 -> WAITLISTED: No seats left in TEST101. Student S14 added to waitlist.
pool-2-thread-18 -> WAITLISTED: No seats left in TEST101. Student S18 added to waitlist.
pool-2-thread-15 -> WAITLISTED: No seats left in TEST101. Student S15 added to waitlist.
pool-2-thread-17 -> WAITLISTED: No seats left in TEST101. Student S17 added to waitlist.
pool-2-thread-20 -> WAITLISTED: No seats left in TEST101. Student S20 added to waitlist.
pool-2-thread-16 -> WAITLISTED: No seats left in TEST101. Student S16 added to waitlist.
pool-2-thread-19 -> WAITLISTED: No seats left in TEST101. Student S19 added to waitlist.
pool-2-thread-1 -> SUCCESS: Registration{student=S1, course=TEST101, seat=1, at=2026-09-06T03:45:09.277594400}
pool-2-thread-2 -> SUCCESS: Registration{student=S2, course=TEST101, seat=2, at=2026-09-06T03:45:09.278607300}
pool-2-thread-3 -> SUCCESS: Registration{student=S3, course=TEST101, seat=3, at=2026-09-06T03:45:09.278607300}
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.065 s -- in com.registration.RegistrationServiceTest

Results:

Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
```

# TESTING APPROACH

Testing focused on validating the system's core correctness guarantee — that seat allocation remains accurate under concurrent access — using JUnit 5.

- Test 1:  
onlyTotalSeatsCountOfRegistrationsSucceedUnderConcurrency
- 20 student threads are submitted to a fixed thread pool, all targeting a course with only 3 seats.

Assertion: exactly 3 registrations succeed.

Assertion: exactly 3 seats show as allocated (no seat is ever double-booked).

Assertion: the remaining students end up on the waitlist.

- Test 2: studentCannotRegisterTwiceForSameCourse
- Verifies a student's first registration attempt succeeds without throwing.

Verifies a second attempt by the same student for the same course throws DuplicateRegistrationException.

Both tests pass consistently across repeated runs, confirming the synchronized block correctly prevents race conditions regardless of thread scheduling order

# CHALLENGES FACED

- Choosing the right lock granularity — synchronizing on the whole `RegistrationService` would have serialized registrations across all courses unnecessarily; synchronizing per-Course object was needed to allow real parallelism across different courses.
- Reliably demonstrating a race condition — because Java threads execute very quickly, some interleavings needed a thread pool large enough (and enough competing students) to reliably expose the race in testing.
- Environment setup — resolving a Java runtime version mismatch (JDK 17 vs. locally installed JDK 21) and a GitHub authentication/permission conflict between two accounts while pushing commits.
- Structuring commits — organizing the implementation into logical, incremental Git commits (models, exceptions, service layer, persistence, reporting, tests) rather than one large commit, to reflect a realistic development process.

# LEARNINGS & KEY TAKEAWAYS

- Hands-on understanding of how Java's synchronized keyword prevents race conditions, and why lock granularity matters for both correctness and performance.
- Practical experience using `ExecutorService` to manage a pool of threads instead of manually creating and joining `Thread` objects.
- Reinforced how custom checked exceptions make failure paths explicit and force callers to handle them, improving code reliability.
- Learned to integrate JDBC persistence and file I/O reporting alongside core business logic in a clean, layered structure.
- Gained experience with the full Git workflow — incremental commits, resolving authentication issues, and pushing changes to a hosted repository.

# FUTURE ENHANCEMENTS

- Add a simple interactive CLI menu so a user can add courses and register students without editing code.
- Support course capacity changes at runtime and multiple administrators.
- Add a REST API layer (e.g., Spring Boot) to expose registration as a web service.
- Add configurable notification (e.g., email/log) when a waitlisted student is promoted to a confirmed seat.
- Extend reporting with summary statistics, such as busiest courses or average wait time.

# REFERENCES

- Oracle Java Documentation — Concurrency (java.util.concurrent), <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- Oracle Java Documentation — JDBC Basics, <https://docs.oracle.com/javase/tutorial/jdbc/basics/>
- SQLite JDBC Driver (xerial/sqlite-jdbc) – <https://github.com/xerial/sqlite-jdbc>
- JUnit 5 User Guide– <https://junit.org/junit5/docs/current/user-guide/>
- Project GitHub Repository – <https://github.com/gauri25bai11091-wq/Programming-in-Java-Evaluated-Project>